

Compiler design

Page No.	
Date	

→ What is a Compiler?

⇒ A compiler is a software program that converts a program written in a high-level language (such as C, C++, or Java) into assembly level language so that ~~the~~ after the some phases computer can understand and execute it.

In simple words,

A computer understands only machine language (0s and 1s).

But human write programs in languages like C, C++, Java because they are easier to read and write.

Since the computer cannot understand these language directly, we need a compiler.

The compiler acts as a **translator** b/w the programmer and the computer.

- Programmer writes the program in a high-level language.
- The compiler translates it into assembly level language.
- Passing more phases after compiler like assembler, linker, loader.
- The CPU executes the machine code.

Eg; `#define pi 3.14;`
`#include <stdio.h>`
`int main() {`
`int a = 3 * pi;` ←
`printf("%d", a);`
`}`

substitute

here,

`#` → preprocessor directives

The preprocessor is a program that runs before the compiler.

- It processes all lines that begin with #.
- It removes comments.

Now,

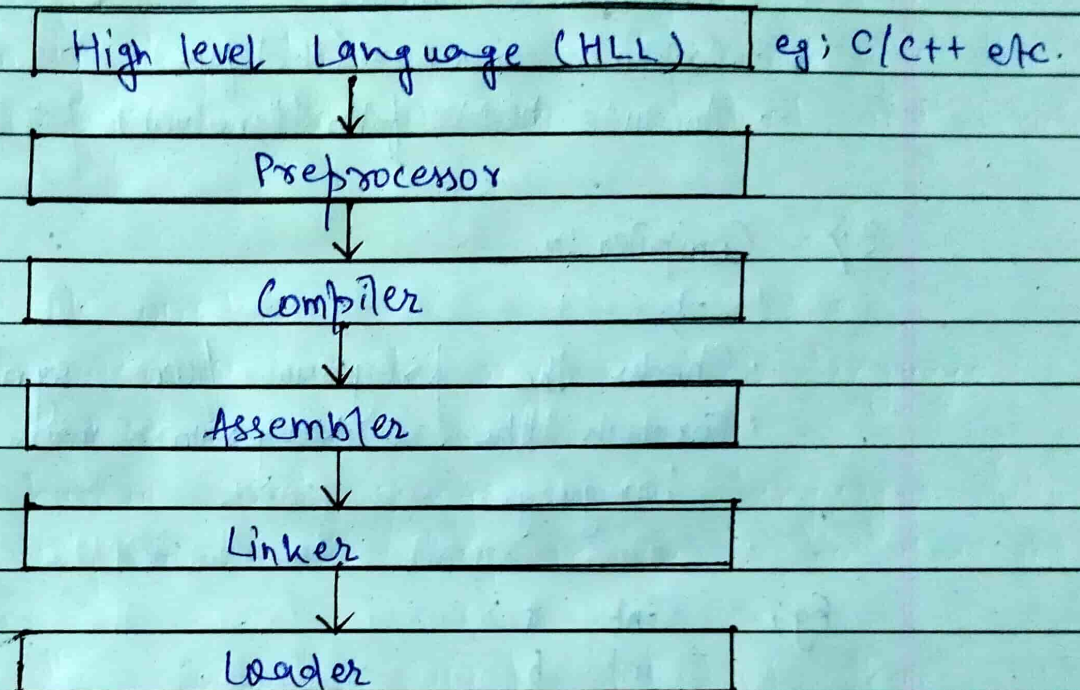
`include <stdio.h>` means

In this program we use a `printf` function and ~~this is not define~~ this ~~function is not define~~ function is not define, so we use `stdio.h` where this function is defined. So we have not written the code for function `printf`, so what will happen to us, we have to include it in this program. So that's why we write this line.


```
int main() {
    int a = 7;
}
```

This program
runs without using header file.

→ Converting a program into an executable program:



1.4 High level language:-

```
#include <stdio.h>
int main() {
    printf("Hello World");
    return 0;
}
```

This program is easy for human to read but the ~~computer~~ CPU cannot understand it directly.

2.4 Preprocessor :-

- Processes all line beginning with #.
- Removes comments.
- Expands macros (#define).
- Include header files (#include)

3.4 Compiler :-

- Checks the program for syntax errors.
- Converts the C program into assembly language.

Eg; int a;
 int b;
 a = a + b * a;

→ Assembly language :

MUL R0, R1
ADD R2, R0
MOV a, R2

4.7 Assembler :-

Converts assembly language into Machine code (Object code).

Assembly :

MOV R1, R2

becomes :-

10110000100

Extension :-

.o or .obj

5.7 Linker :-

The object file cannot run because function like printf() are stored in libraries.

Working :

- Combines our object file with required library files that we used in a program.
- Resolves function references.

Eg; If our program uses printf() function

Date

The linker find the actual `printf()` code from the C standard libraries and attaches it.

Extension :-

`.exe` (executable file)

`0.6` Loader :-

- loads the executable program from the hard disk into RAM.
- Gives control to the CPU.
- Starts execution from the `main()` function.

After loading, our program runs.

Hello World ~~time~~
is displayed on the screen.

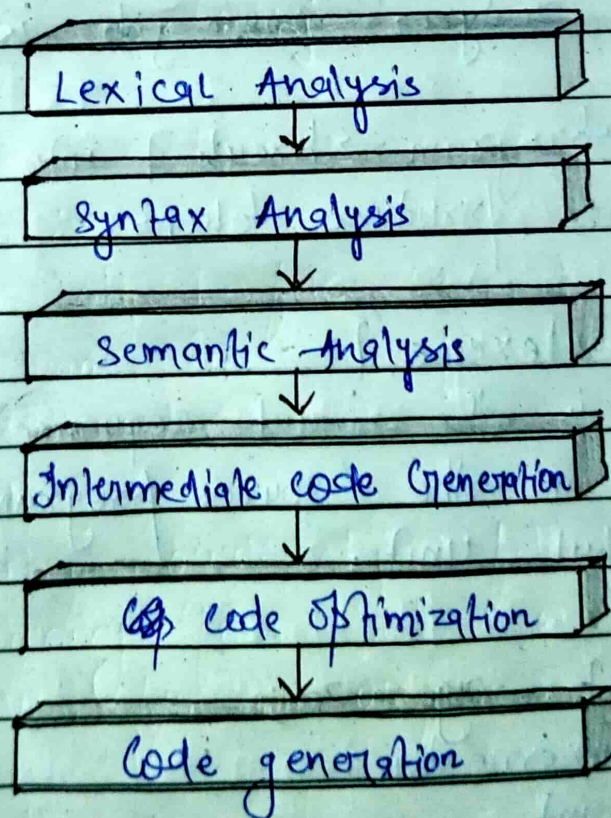
~~beginning~~

→ Phases of Compiler :-

A compiler converts a program written in high-level language (like C or C++) into assembly level language and checks the program for syntax error.

It does not do this in one step. Instead, it completes the work in 6 phases.

→ Diagram of Compiler Phases :-



→ Phase 1: Lexical Analysis :-
Lexical Analysis is the first phase of a compiler.

It reads the **source program** characters **by character** and group those characters into meaningful ~~for~~ units called tokens.

The component that perform lexical analysis is called a lexical Analyzer or **Scanner**.

In simple words,

The lexical analyzer's main job is :

Take the program → break it into smaller meaning pieces → produce tokens.

→ What is lexeme?

A lexeme is the actual sequence of characters present in the source program that forms a meaningful unit.

Eg: int age = 20;

The lexeme are:

int

age

=

20

;

These are the actual pieces found in the source code.

→ What is a token?

A token is the category or classification given to a lexeme.

int age=20;

lexeme

Token

int

keyword

age

identifier

=

assignment operator

20

constant

;

separator

Lexeme $\Rightarrow$ actual text

Token $\Rightarrow$ category of that text

→ Main types of Tokens :-

(i) Keywords :-

Keywords are reserved words that have a predefined meaning in a programming language.

Eg;

int, float, char, if, else, while, for, return etc.

Eg;

```
int age;
```

here,

int → keyword.

we cannot normally use int as a variable name because it is reserved.

(ii) Identifiers :-

Identifiers are the names given by the programmer to program elements.

Eg;

```
int age;  
int arr[];  
void fun() {  
    // body  
}
```

here,

age → Identifier

arr → Identifier

fun → Identifier

Identifier can be used for:

- variables
- Functions
- Array
- Structures
- etc,

(iii) Constant / Literals :-

A constant is a value that appears directly in the program.

Eg; 10, 25, 3.14, 'A', "Hello"

Eg;

```
int age = 20;
```

here,

20 → constant

(iv) Operators :-

operator perform operation on values.

Eg;

+ , - , * , / , % , = , == , > , < , << , >> , ||

Eg;

a + b

here,

a → Identifier

+ → Operator

b → Identifier

(v) Separator/punctuator :-

These are symbols used to separate or structure parts of a program.

Eg;

; , , (,) , { , } , [,]

Eg;

int a = 5 ;

here,

; → indicates the end of statement.

(vi) Strings :-

Strings are nothing but an array of character ended with a null character ('\\0').

printf ("Hello World");

↳ String.

Eg:

$$i \rightarrow n \rightarrow t \rightarrow \text{space} \rightarrow x \rightarrow \text{space} \rightarrow = 1 \rightarrow 0 \rightarrow ;$$

2.4 Create Tokens :-

int $\rightarrow$ Keyword
x $\rightarrow$ Identifier
10 $\rightarrow$ Constant

Remove white spaces in
spaces, tabs, and newlines generally don't affect
the meaning of c code.

for eg;

```
int x = 10;
```

and

```
int    x = 10;
```

means the same thing.

→ The lexical analyzer generally ignores these unnecessary spaces.

4.6 Removal of comments :-

Comments are written for humans and normally aren't needed ~~for~~ by later compiler phases.

```
// This is a comment
```

```
int x = 10;
```

The comment is removed during lexical processing rather than being treated as a meaningful program token by the later phase.

~~12/12/2021~~
5. Detects lexical errors:-

The lexical analyzer also identifies characters or sequences that cannot form valid tokens.

Eg;

int @x;

If @ is not valid in the language's identifier/operator rules, the lexical analyzer reports an error.

lexical error

Eg;

int 12abc;

This is invalid as an identifier in C because as an identifier cannot start with a digit.

6. Symbols Table Interaction:-

The lexical analyzer also works with the symbol table.

A symbol table stores information about identifiers.

Eg;

```
int age;  
float salary;
```

The compiler may store information such as:

<u>Name</u>	<u>type</u>
age	int
salary	float

The lexical analyzer can recognize age and salary as identifier, while later compiler phases use the symbol-table information such as type and ~~scope~~ scope checking.

→ Input and output of lexical Analyzer :-

Input :-

Source program

```
int a = 10;
```

Processing :-

Characters



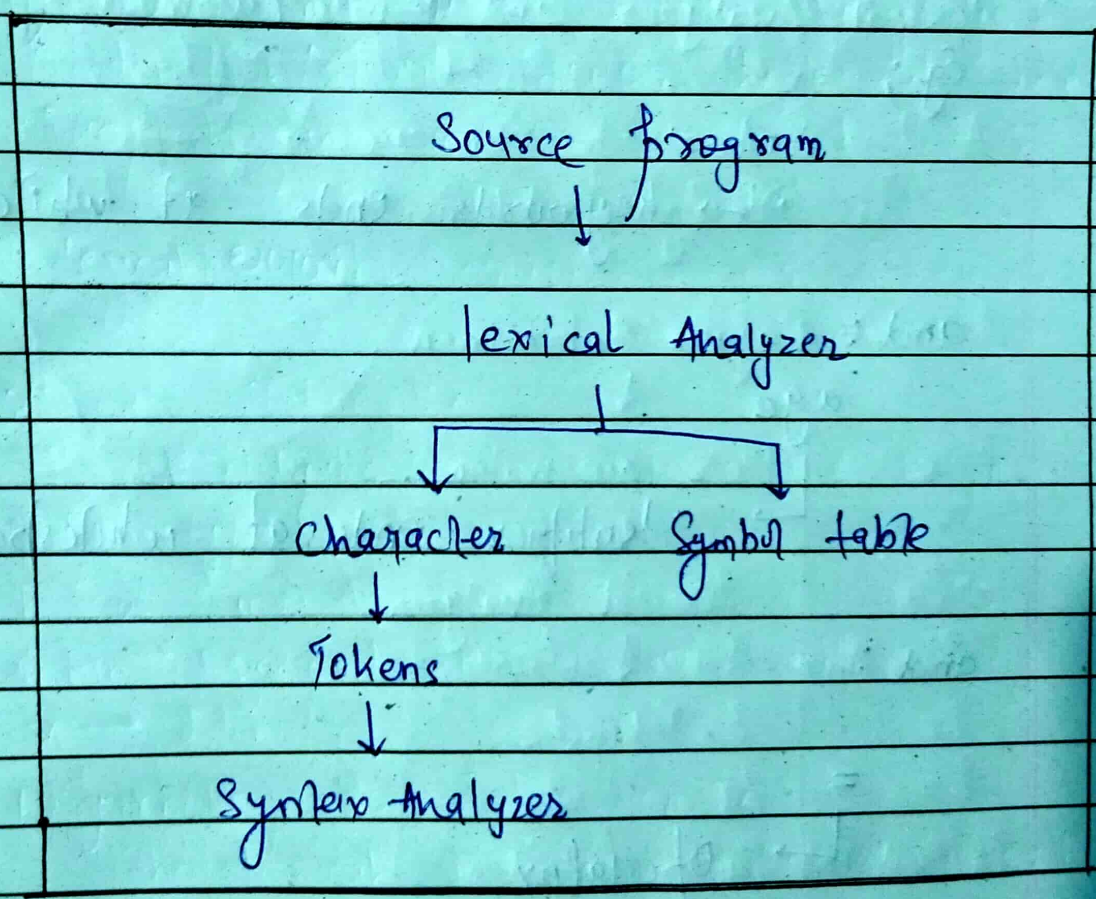
lexemes



Tokens

Output :-

- < keyword, int >
- < identifier, a >
- < assignment, = >
- < constant, 10 >
- < separator, ; >



→ How does it know where one Token Ends?

Eg;

int age = 21;

The lexical analyzer uses rules of the programming language to recognize boundaries b/w tokens.

Eg;

int

↳ keywords ends at whitespace

and:

age

↳

identifier ends at whitespace

and:

=

↳

operator

and:

20

↳

constant ends before ;

These rules are described using concepts such as regular expression and finite automata.

→ lexical Analyzer and Regular Expression :-

In compiler design, lexical structures are commonly describe using regular expression.

Eg;

an identifier might follow ~~as~~ a rule such as :-

letter (letter | digits)* → Regular expression

Means:-

- first character → letter
- After that → zero or more letters/digits

Eg;

Age
marks
Student
Total123

are valid according to this simplified rule

But :

123student

is not valid as a C identifier because it starts with a digit.

→ Lexical Analyzer and Finite Automata:-

A lexical analyzer can use a **Finite automaton** to recognize tokens.

Regular Expression



Finite Automaton



Lexical Analyzer



Tokens

→ Define DFA (deterministic finite automaton) with example?

⇒ A DFA is a machine (model) used to check whether a given string is accepted or rejected based on a fixed rules.

In simple words;

A DFA is a type of F.A where for every state and input symbol, there is exactly one ~~state~~ next state.

formal definition:-

- $Q \rightarrow$ set of states
- $\Sigma \rightarrow$ input symbol
- $q_0 \rightarrow$ initial state
- $F \rightarrow$ set of final state
- $\delta \rightarrow$ Transition func.

Q. Construct a DFA which accept the set consists of all strings with no more than three a's where i/p alphabet is $\Sigma = \{a, b\}$.

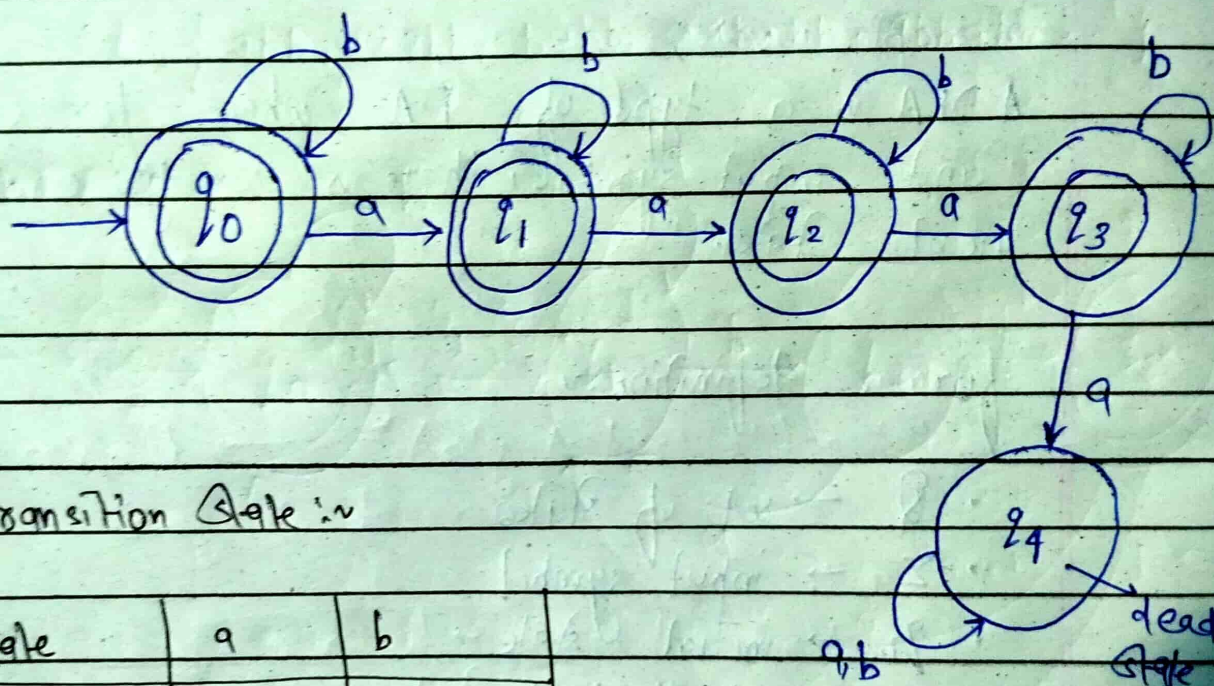
⇒ Given:

Input alphabet
 $\Sigma = \{a, b\}$

$L = \{\text{Strings with no more than three a's}\}$

$= \{ \epsilon, a, aa, a^3, aab, aabb, abbb, abba, baab, \dots \}$

DFA :-



Transition Table :-

State	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_2
q_3	q_4	q_3
q_4 (dead state)	q_4	q_4

Q. Construct a DFA corresponding to regular expression

$$1^*(10)^*$$

$\Rightarrow$ Given:

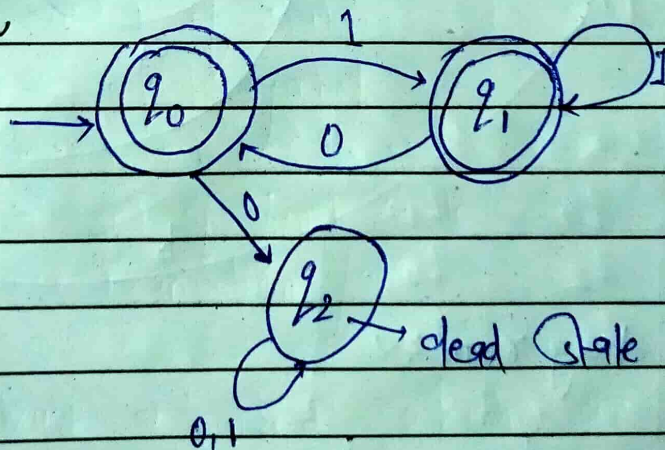
$$\text{Regular expression} = 1^*(10)^*$$

$$L = \{ \epsilon, 1, 10, 11, 111, \dots, 10101010, \dots \}$$

means it will end with 1 or 10

$$L \neq \{ 0, 100, \text{Starting with 0} \}$$

DFA :-



Transition table:-

State	0	1
$\rightarrow q_0$	q_2	q_1
q_1	q_0	q_1
q_2 (dead state)	q_2	q_2

P18:

Q Convert regular expression $(a+b)^*abb$ into DFA.

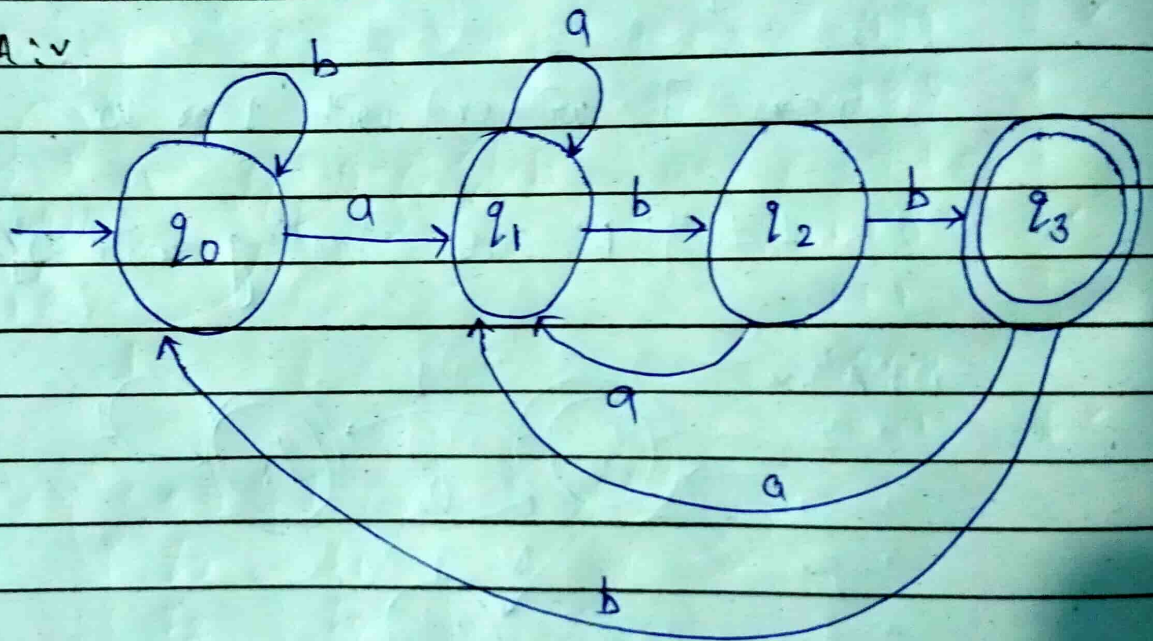
⇒ Given:

Regular expression:

$$(a+b)^*abb$$

$$L = \{ abb, aabb, babb, aababb, abbaabb, \dots \}$$

DFA:~



Transition diagram:~

State	a	b
q ₀	q ₁	q ₀
q ₁	q ₁	q ₂
q ₂	q ₁	q ₃
q ₃	q ₁	q ₀

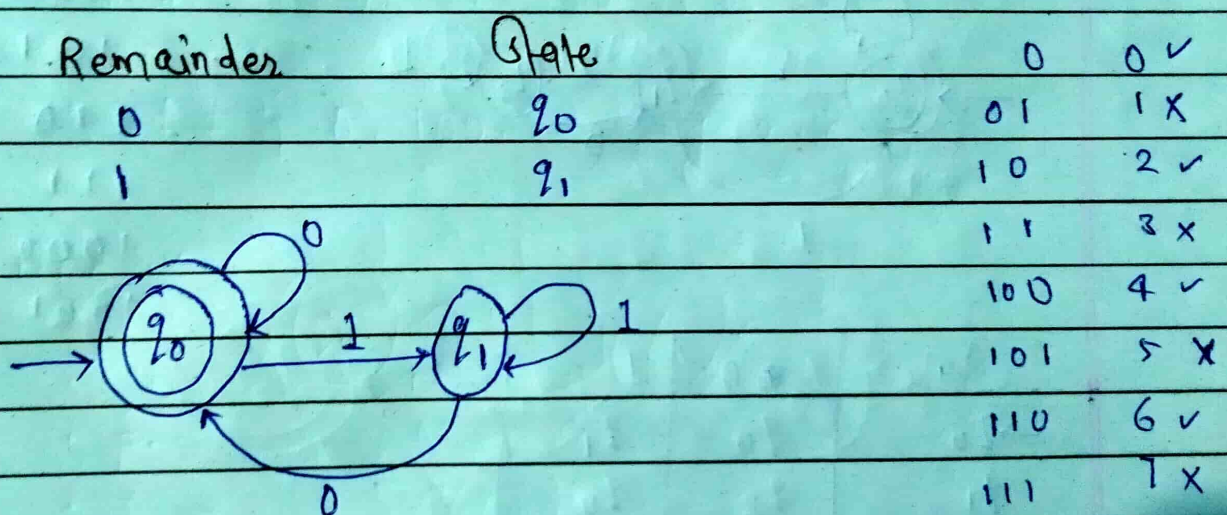
Q. Construct a DFA which accept the set of all binary string that interpreted as binary representation of an unsigned decimal integer, is divisible of 2.

=>

Given:

divisible by 2

$L = \{ 0, 10, 110, 100, 1000, \dots \}$



Transition table:

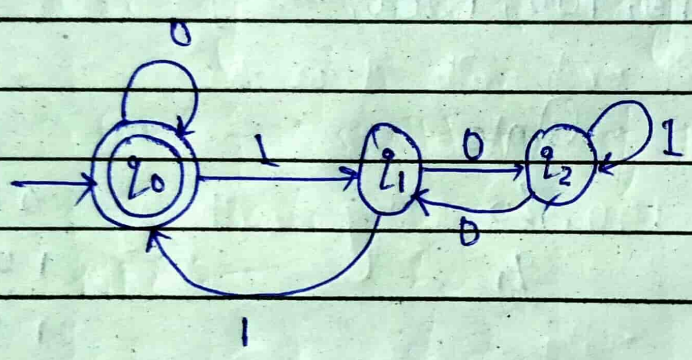
State	0	1
→ q_0	q_0	q_1
q_1	q_0	q_1

Q. Same as before, just divide by 3.

$$R.E = \{ 0, 11, 110, 1001, \dots \}$$

Remainder State

0 q_0
 1 q_1
 2 q_2



0 0 ✓
 01 1
 10 2
 11 3 ✓
 100 4
 101 5
 110 6 ✓
 111 7
 1000 8
 1001 9 ✓

State	0	1
→ q_0	q_0	q_1
q_1	q_2	q_0
q_2	q_1	q_2

Q. Same as before, just divide by 5 :v

$$R.E = \{0, 101, 1010, \dots\}$$

Remainder State

0 q_0

1 q_1

2 q_2

3 q_3

4 q_4

0 0 ✓

1 1

10 2

11 3

100 4

101 5 ✓

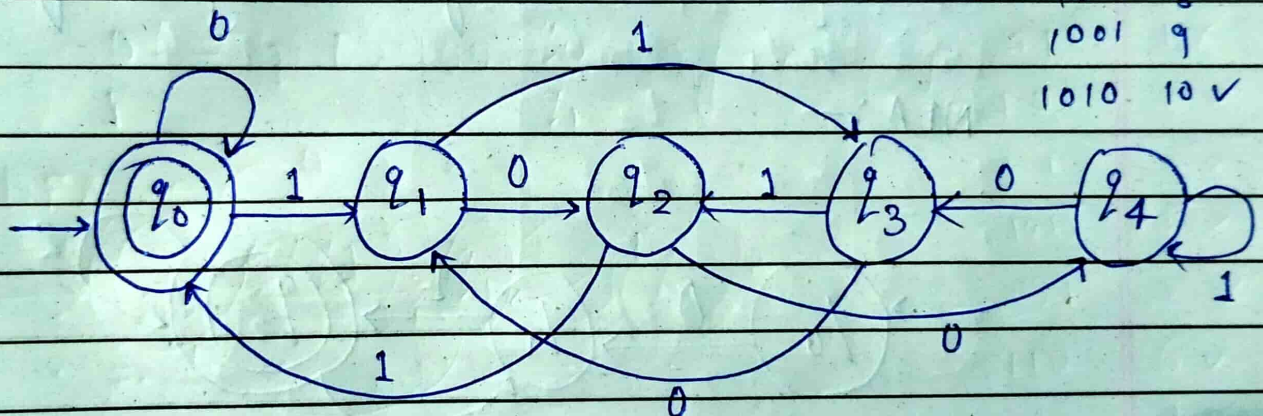
110 6

111 7

1000 8

1001 9

1010 10 ✓



Transition table :v

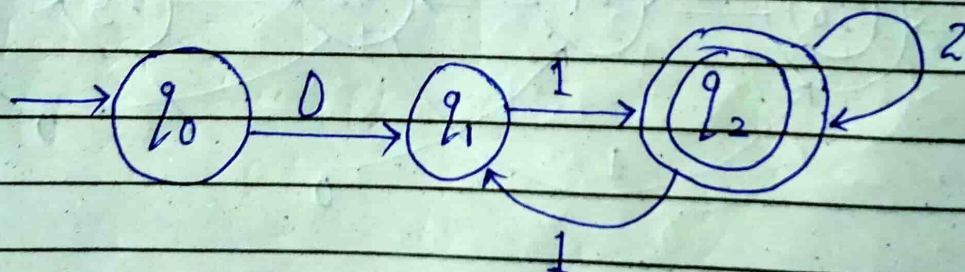
State	0	1
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_0
q_3	q_4	q_2
q_4	q_3	q_4

→ Define NFA (Non-deterministic finite automaton):
 A NFA is a finite automaton where a state and ~~an~~ input symbol, there can be multiple possible next state. (or none).

Eg: Construct a NFA that will accept strings of 0's, 1's and 2's beginning with a 0's followed by odd number of 1's and ending with any number of 2's.

⇒ R.E = { 0's, odd no of 1, any no. of 2's }

NFA :-



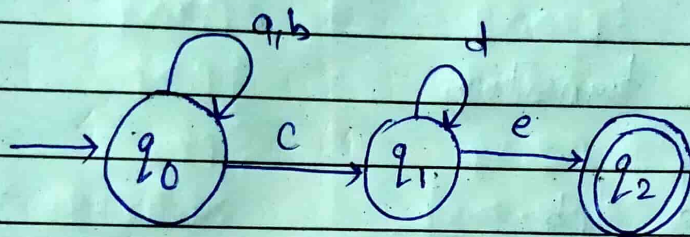
State transition :-

State	0	1	2
→ q ₀	q ₁	-	-
q ₁	-	q ₂	-
q ₂	-	q ₁	q ₂

Q. Construct a finite automata corresponding to the regular expression:

$\Rightarrow$ Given, $(a+b)^* cd^* e$

RE = $\{ ce, ace, bce, abcde, \dots \}$



State transition :-

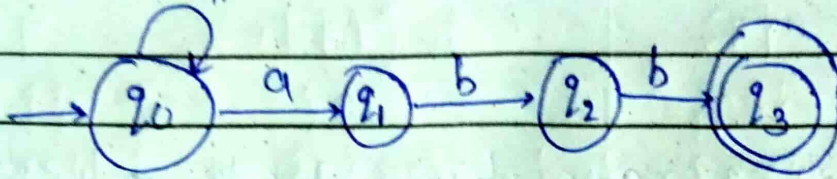
State	a	b	c	d	e
$\rightarrow q_0$	q_0	q_0	q_1	-	-
q_1	-	-	-	q_1	q_2
q_2	-	-	-	-	-

Q10-1024

Q Convert regular expression $(a+b)^* abb$ into NFA.
 Given.

$(a+b)^* abb$

a, b



State transition

State	a	b
q ₀	q ₀ , q ₁	q ₀
q ₁	-	q ₂
q ₂	-	q ₃
q ₃	-	-

→ Convert NFA → DFA :-

Steps :

1. Draw transition table
2. With the help of transition table, take one state
3. find each transition of each state
4. Repeat until no new state
5. find final state:
Any DFA state containing NFA final state → DFA final state.

Q.1. Construct a DFA equivalent to the NFA given by $M = \{ \{P, Q, R, S\}, \{0, 1\}, \delta, P, \{S\} \}$ where δ is defined in the following table.

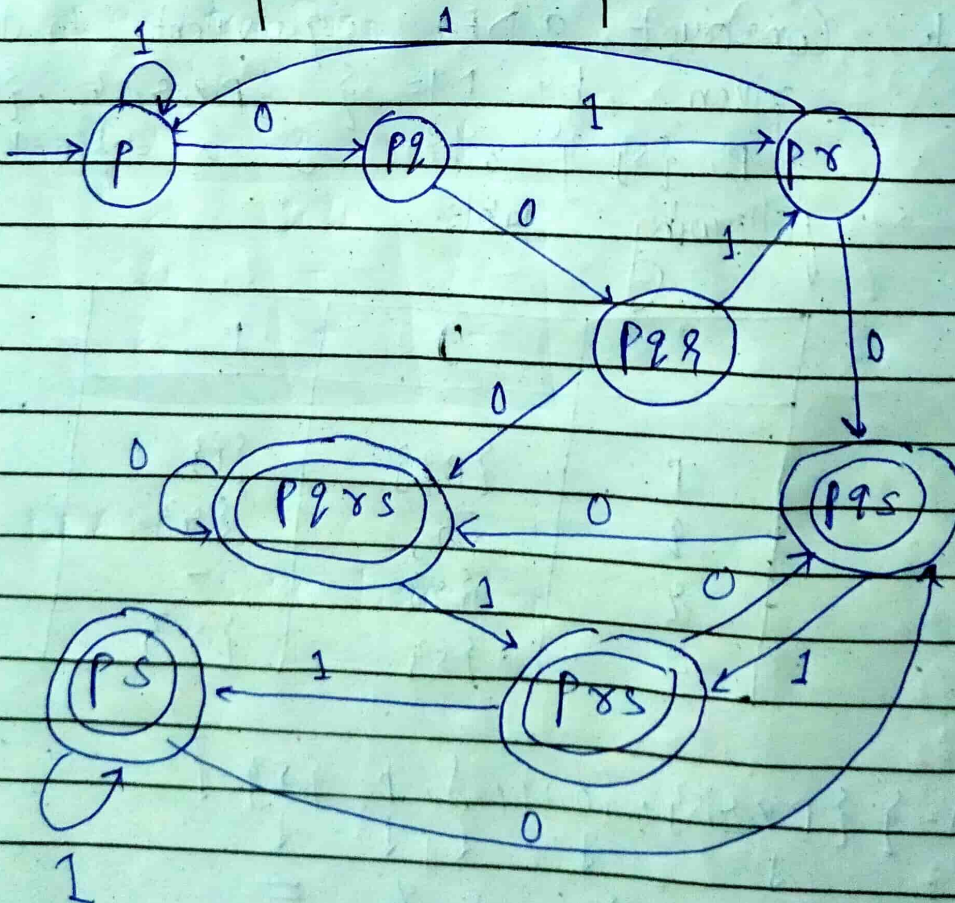
	0	1
P	$\{P, Q\}$	$\{R\}$
Q	$\{R\}$	$\{R\}$
R	$\{S\}$	-
S	$\{S\}$	$\{S\}$

⇒ $M = \{ \{P, Q, R, S\}, \{0, 1\}, \delta, P, \{S\} \}$

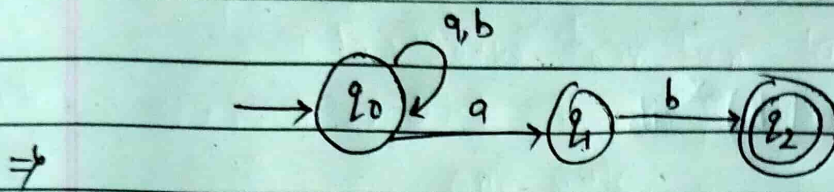
$\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 δ Σ δ q_0 F

State transition :-

Qate	0	1
$\rightarrow P$	$\{P, 2\}$	$\{P\}$
$\{P, 9\}$	$\{P, 9, 2\}$	$\{P, 9\}$
$\{P, 9, 8\}$	$\{P, 9, 8, 5\}$	$\{P, 8\}$
$\{P, 9, 8, 5\}$	$\{P, 9, 8, 5\}$	$\{P, 8, 5\}$
$\{P, 8, 5\}$	$\{P, 9, 5\}$	$\{P, 5\}$
$\{P, 8\}$	$\{P, 9, 5\}$	$\{P\}$
$\{P, 9, 5\}$	$\{P, 9, 8, 5\}$	$\{P, 9, 8, 5\}$ $\{P, 9, 5\}$
$\{P, 5\}$	$\{P, 9, 5\}$	$\{P, 5\}$



Q. Construct DFA from the following NFA transition diagram.

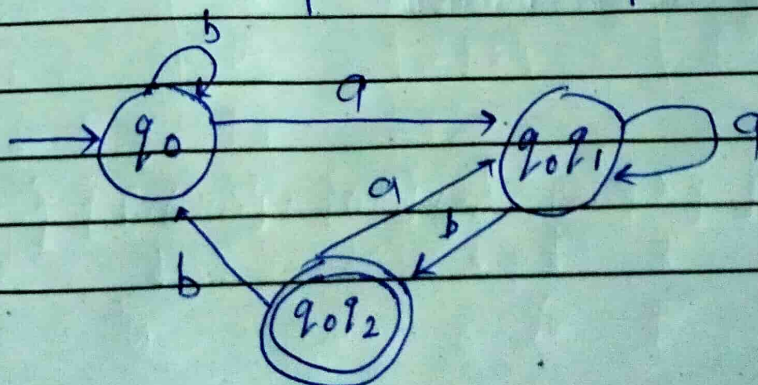


NFA transition table

State	a	b
q_0	q_0, q_1	q_0
q_1	-	q_2
q_2	-	-

DFA transition table

State	a	b
q_0	q_0, q_1	q_0
$q_0 q_1$	$q_0 q_1$	$q_0 q_2$
$q_0 q_2$	$q_0 q_1$	q_0



P18

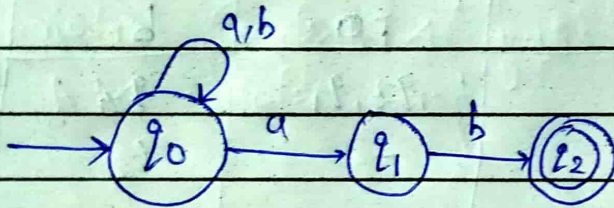
Q Convert the regular expression $(a|b)^* ab$ into NFA, then convert NFA into minimal DFA.

⇒ Given:

$$(a|b)^* ab$$

$$R.E = \{ ab, aab, baab, \dots \}$$

NFA :-



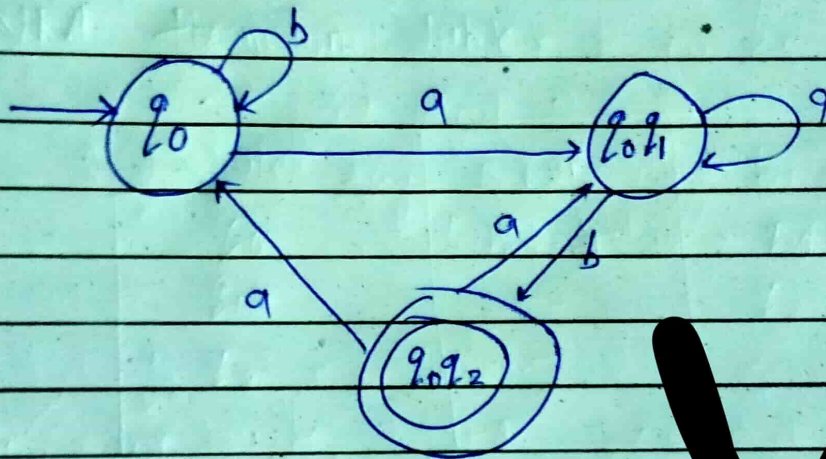
State transition

State	a	b
q ₀	q ₀ q ₁	q ₀
q ₁	-	q ₂
q ₂	-	-

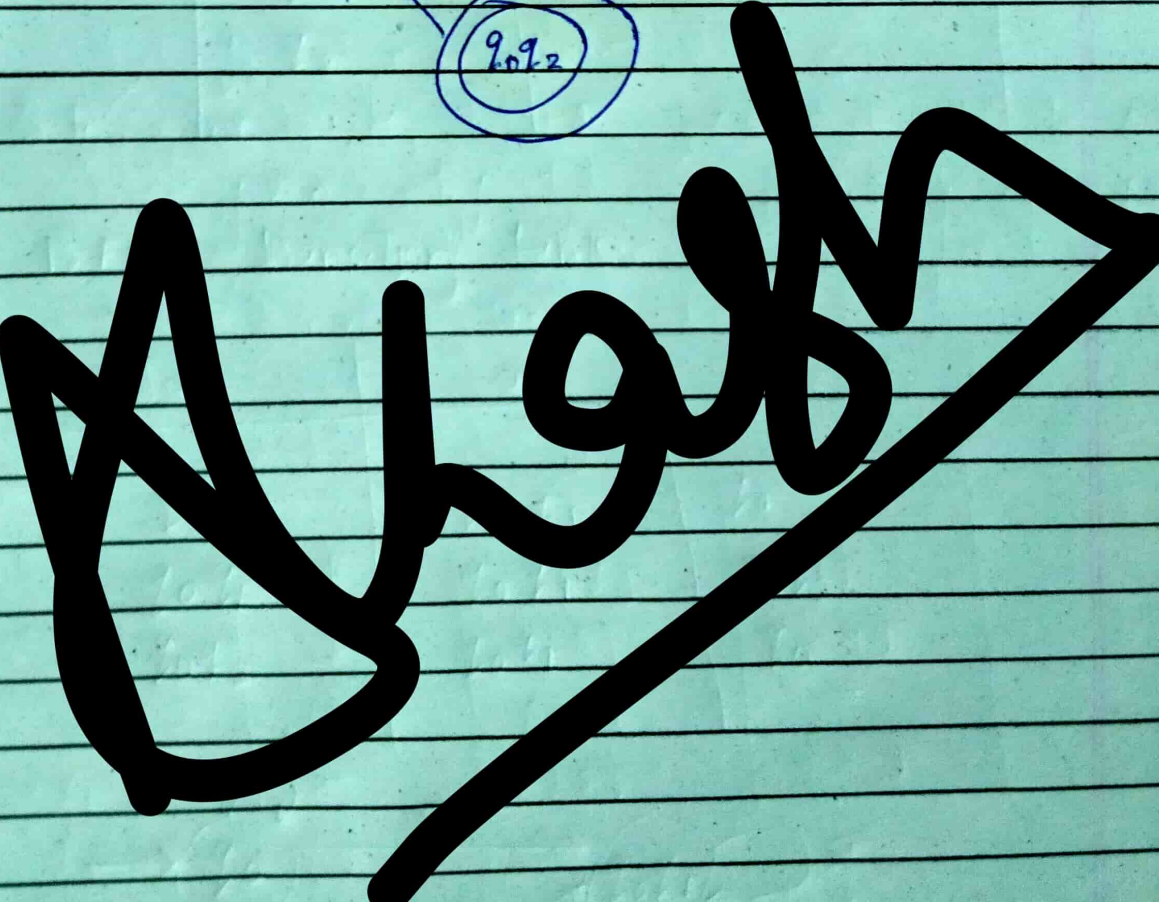
DFA :-

State transition

State	a	b
q_0	$q_0 q_1$	q_0
$q_0 q_1$	$q_0 q_1$	$q_0 q_2$
$q_0 q_2$	$q_0 q_1$	q_0



9.9.2

A large, thick, black scribble or signature that covers most of the page. It appears to be a stylized, abstract drawing or a very heavy, illegible signature. The scribble starts with a large, looped shape on the left, followed by a series of thick, sweeping strokes that extend across the middle and right side of the page. A long, straight, thick line runs diagonally from the bottom left towards the middle right, intersecting the other strokes. The overall effect is a dense, black, abstract mark that obscures the underlying text and lines of the notebook paper.